Prof. KHALD

Développement JEE

1

Rappel sur la POO

- Classes
- Objets
- Héritage
- Polymorphisme
- Surcharge
- Interface
- Classes abstraites
- Exception
- Threads
- Annotations

- Machine Vrituelle
- Les fichiers .class
- Garbage Collector - Références Java
- Java Beans
- JIT Compiler (Just-In-Time)
- Heap et Stack
- Couplage
- Serialization
- Stateless / Stateful
- IoC & Injection de dépendances

2

```java
public void demo() {
    int x = 10; // Stack
    Person p = new Person(); // p → Stack, objet → Heap
}
```

- **La JVM exécute le bytecode Java et gère automatiquement la mémoire et les threads.**
- **Un fichier .class contient le bytecode généré à partir du code Java et exécutable par la JVM.**

- Stack → variables locales, appels de méthodes
- Heap → objets
- La référence est dans le stack, l'objet dans le heap

3

Garbage Collector & Références Java

```java
Person p1 = new Person();
Person p2 = p1;    // deux références vers le même objet
p1 = null;         // l'objet est encore référencé
p2 = null;         // plus aucune référence → GC possible
```

- Un objet est détruit **s'il n'est plus référencé**.

Le Garbage Collector libère automatiquement la mémoire des objets qui ne sont plus référencés.

Une référence Java est une variable qui pointe vers un objet stocké dans le heap.

4

JavaBeans

```java
public class Student implements Serializable {

    private String name;

    public Student() {} // constructeur vide obligatoire

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
```

- attributs privés
- getters / setters
- constructeur sans argument

Un JavaBean est une classe simple avec attributs privés, constructeur vide et getters/setters.

5

JIT Compiler (Just-In-Time)

```java
for (int i = 0; i < 1_000_000; i++) {
    calcul();
}
```

- Le JIT détecte le code souvent exécuté
- Il le compile en **code machine**
- Meilleures performances

Le compilateur JIT transforme à l'exécution le bytecode fréquemment utilisé en code machine pour améliorer les performances.

6

Couplage fort & faible

moins de `new` = moins de couplage

```
class Service {
    Repository repo = new Repository(); // dépendance directe
}
```

```
class Service {
    Repository repo;

    Service(Repository repo) {
        this.repo = repo;
    }
}
```

7

Sérialisation

```
ObjectOutputStream out =
    new ObjectOutputStream(new FileOutputStream("data.ser"));
out.writeObject(student);
out.close();

ObjectInputStream in =
    new ObjectInputStream(new FileInputStream("data.ser"));
Student s = (Student) in.readObject();
```

Transformer un objet → flux (fichier / réseau)

8

Stateful

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }
}
```

Stateless

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
}
```

- Stateful → conserve un état
- Stateless → résultat dépend uniquement des paramètres

9

IoC & Injection de dépendances

- L'objet **ne crée pas ses dépendances**
- Elles sont **injectées**

L'injection de dépendances consiste à fournir les objets nécessaires à une classe au lieu de les créer avec new.

```
class OrderService {

    private Payment payment;

    public OrderService(Payment payment) {
        this.payment = payment;
    }
}

Payment p = new CardPayment();
OrderService service = new OrderService(p);
```

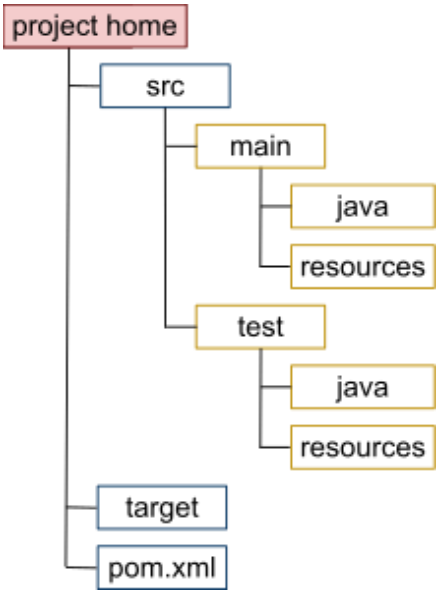
10

Résumé

Le Stack exécute, le Heap stocke, le GC nettoie, SOLID structure, et l'Injection libère le code.

11

Projet Maven



12

Projet maven

Un projet Maven est :

- un projet Java standardisé
- la structure des dossiers est imposée,
- les dépendances sont gérées automatiquement,
- la compilation, les tests et le packaging sont automatisés.

Maven évite :

- les JAR copiés à la main,
- les configurations compliquées,
- les builds non reproductibles.

13

- Le fichier **pom.xml** (Project Object Model) décrit tout le projet :
  - nom du projet
  - version
  - dépendances
  - plugins
  - type de packaging (JAR, WAR…)

14

pom.xml

Maven :

1. télécharge la librairie
2. télécharge ses dépendances
3. les stocke dans le **repository local**
4. les ajoute automatiquement au projet

```
<project>
<modelVersion>4.0.0</modelVersion>
<groupId>ma.ecole</groupId>
<artifactId>demo-maven</artifactId>
<version>1.0</version>
<dependencies>
 <dependency>
 <groupId>org.junit.jupiter</groupId>
 <artifactId>junit-jupiter</artifactId>
<version>5.10.0</version>
 <scope>test</scope>
 </dependency>
</dependencies>
</project>
```

15

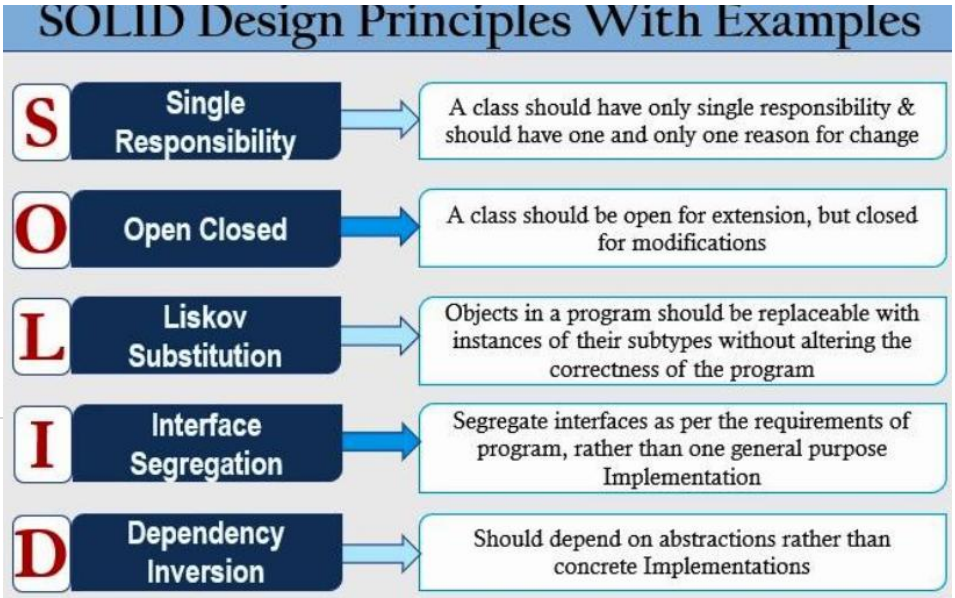
Rappel sur les Bonnes pratiques de développement Java

- SOLID :

- **SOLID** est un ensemble de **5 principes de conception logicielle** qui permettent de :
- réduire le **couplage**
- améliorer la **maintenabilité**
- faciliter l'**évolution** et les **tests**
- produire un **code propre et robuste**

16

SOLID



17

S – single Responsablity

- Une classe ne doit avoir qu'une seule raison de changer.

```
class UserService {
    void saveUser() {}
    void sendEmail() {}
}
```

```
class UserService { void saveUser() {} }
class EmailService { void sendEmail() {} }
```

18

O - Open / Closed Principle (OCP)

```java
interface Paiement {
    void payer(double montant);
}
class PaiementCB implements Paiement {
    public void payer(double montant) { }
}
class PaiementPaypal implements Paiement {
    public void payer(double montant) { }
}
```

```java
class PaiementService {
    void payer(String type) {
        if (type.equals("CB")) { }
        else if (type.equals("PayPal")) { }
    }
}
```

```java
class PaiementService {
    void payer(Paiement paiement, double montant) {
        paiement.payer(montant);
    }
}
```

19

L - Liskov Substitution Principle (LSP)

```java
interface Forme {
    int surface();
}
class Rectangle implements Forme {
    int largeur, hauteur;
    public int surface() {
        return largeur * hauteur;
    }
}
class Carre implements Forme {
    int cote;
    public int surface() {
        return cote * cote;
    }
}
```

Une sous-classe doit pouvoir remplacer sa classe mère sans erreur

```java
class Rectangle {
    void setLargeur(int l) { }
    void setHauteur(int h) { }
}

class Carre extends Rectangle {
    @Override
    void setLargeur(int l) {
        // change aussi la hauteur ✗
    }
}
```

20

I - Interface Segregation Principle (ISP)

les classes n'implémentent que ce qu'elles utilisent.

```
interface Machine {
    void imprimer();
    void scanner();
    void faxer();
}

class ImprimanteSimple implements Machine {
    public void faxer() { throw new UnsupportedOperationException(); }
}
```

```
interface Imprimante {
    void imprimer();
}
interface Scanner {
    void scanner();
}
class ImprimanteSimple implements Imprimante {
    public void imprimer() { }
}
```

21

D – dependency injection

```
interface Database {
    void connecter();
}
class MySQLDatabase implements Database {
    public void connecter() { }
}
class Application {
    private Database database;

    public Application(Database database) {
        this.database = database;
    }
}
```

```
class MySQLDatabase {
    void connecter() { }
}
class Application {
    MySQLDatabase db = new MySQLDatabase();
}
```

testabilité + flexibilité (MySQL, Oracle, Mock…)

22

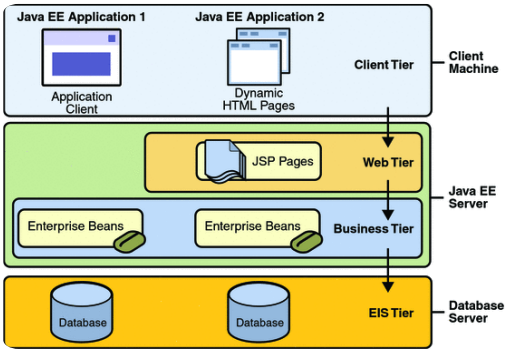
SOLID

Lettre	Principe	Idée clé
S	Single Responsibility	Une classe = un rôle
O	Open / Closed	Étendre sans modifier
L	Liskov Substitution	Remplacement sûr
I	Interface Segregation	Interfaces fines
D	Dependency Inversion	Abstractions

23

Principe	Résumé
SRP	Une classe doit faire **une seule chose**
OCP	Étendre le code **sans le modifier**
LSP	Une sous-classe doit rester **remplaçable**
ISP	Interfaces **petites et ciblées**
DIP	Dépendre des **interfaces**, pas des implémentations

24



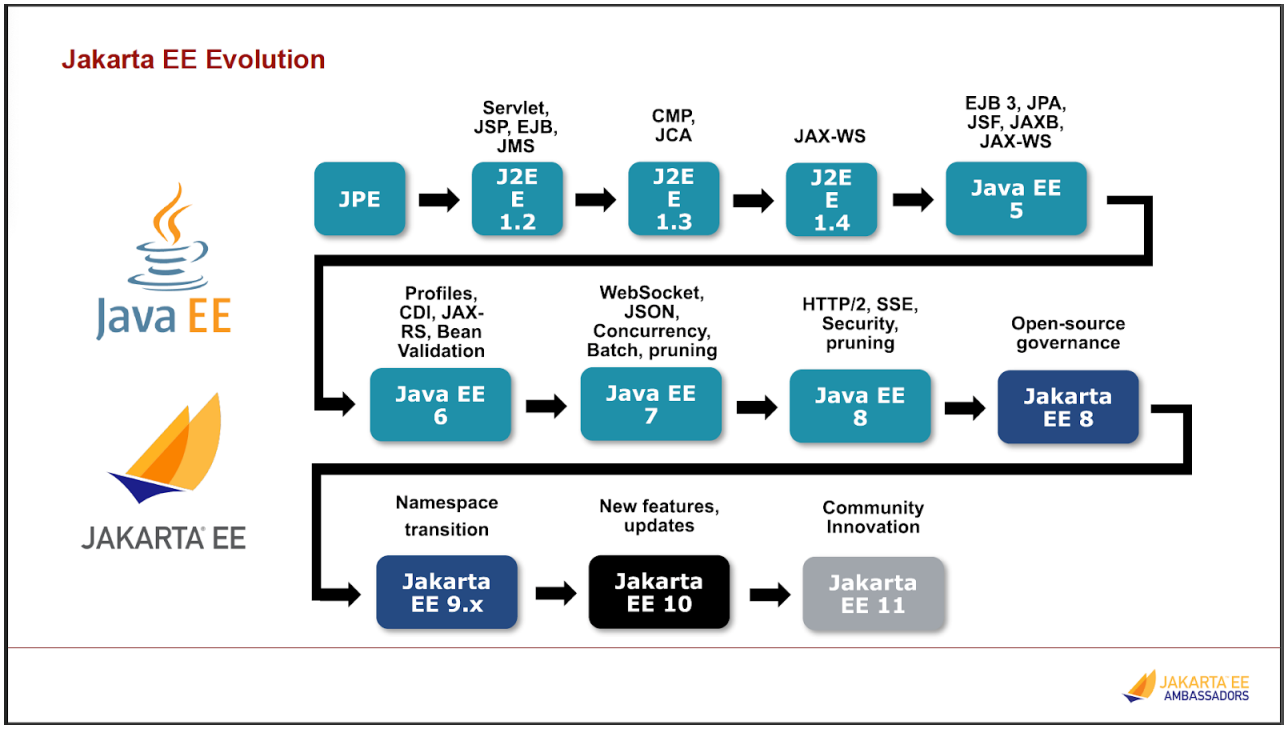
- **JEE** signifie **Java Enterprise Edition**. C'est une **plateforme Java** destinée au développement d'**applications d'entreprise** robustes, sécurisées et distribuées (applications web, systèmes d'information, services métiers).
- Aujourd'hui, JEE est officiellement appelé **Jakarta EE**.

25

Évolution historique

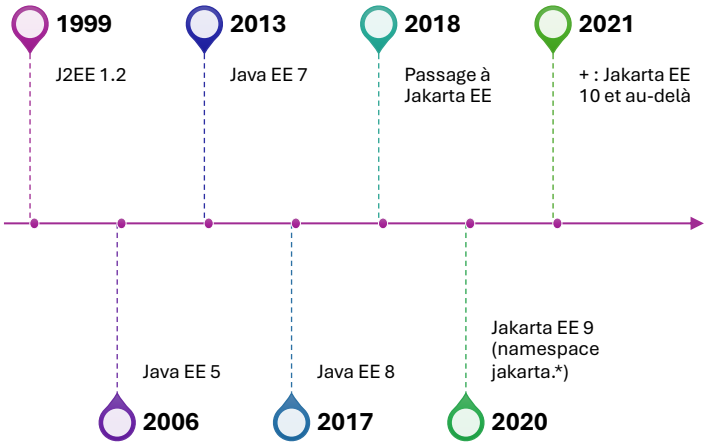
- **J2EE** → ancienne appellation
- **Java EE** → standard Oracle
- **Jakarta EE** → depuis 2019 (géré par la Fondation Eclipse)
- À partir de Jakarta EE 9, les packages passent de javax.* à jakarta.*.

26



27

Historique du JEE



28

À quoi sert JEE ?

- JEE est utilisé pour construire des applications :
  - **Scalables** (support de milliers d'utilisateurs)
  - **Sécurisées**
  - **Transactionnelles**
  - **Multi-tiers** (client / web / métier / données)

29

Architecture JEE

- JEE repose sur une architecture **en couches** :
- **Couche client**
  - Navigateur web, application mobile, client lourd
- **Couche Web**
  - Servlets
  - JSP / JSF
  - REST (JAX-RS)
- **Couche Métier**
  - EJB (Enterprise JavaBeans)
  - Logique métier, transactions
- **Couche Données**
  - JDBC, JPA
  - Bases de données relationnelles

30

Serveurs d'application

Pour exécuter une application JEE, on utilise un **serveur d'applications** :

- WildFly
- GlassFish
- Payara
- JBoss
- WebLogic
- WebSphere

31

Composants clés de JEE

- **Servlets** : gestion des requêtes HTTP
- **JSP** / **JSF** : interfaces web
- **EJB** : logique métier
- **JPA** : persistance des données
- **JAX-RS** : services web REST
- **JMS** : messagerie
- **JAAS** / **Security** : sécurité

32

Java SE ?

La base de Java

- Java SE fournit les fondamentaux du langage Java.

Contenu principal

- Syntaxe Java
- Types, classes, objets
- Collections
- Exceptions
- Threads
- I/O
- JVM

- **Exemples d'applications**
  - Applications console
  - Applications desktop (Swing, JavaFX)
  - Bibliothèques Java

33

Java EE ?

- Une extension de Java SE pour les applications d'entreprise (Web & distribuées)
- Java EE n'est plus utilisé sous ce nom aujourd'hui
- **Contenu principal**
  - Servlets
  - JSP
  - EJB
  - JPA
  - JMS
  - Web Services

- **Exemples d'applications**
  - Applications web
  - Applications bancaires
  - Systèmes d'information

34

JEE / Jakarta EE	Spring
Standard officiel	Framework
Basé sur des spécifications	Plus flexible
Très utilisé dans les grandes structures	Très populaire aujourd'hui
Configuration parfois lourde	Plus léger

35

Spring framework

- Un **framework Java** alternatif à Java EE / Jakarta EE
- **Objectifs**
  - Simplifier le développement
  - Réduire la configuration
  - Favoriser les bonnes pratiques
- **Concepts clés**
  - IoC (Inversion of Control)
  - Injection de dépendances
  - Spring MVC
  - Spring Data
  - Spring Security

36

Spring Boot

Une surcouche de Spring qui rend Spring ultra simple et rapide

Ce que Spring Boot apporte

Configuration automatique	Serveur intégré (Tomcat)	Zéro XML	Démarrage rapide

37

Comparaison

Technologie	*Rôle*	*Complexité*	*Usage principal*
Java SE	Base du langage	⭐	Fondamentaux
Java EE	Entreprise (ancien)	⭐⭐⭐⭐	Applis lourdes
Jakarta EE	Java EE moderne	⭐⭐⭐	Applis d'entreprise
Spring	Framework Java	⭐⭐	Web, API
Spring Boot	Spring simplifié	⭐	Microservices

38

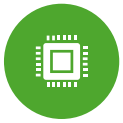
On utilise la JVM pour rendre Java portable, sécurisé, performant et indépendant du système.

PORTABILITÉ – “WRITE ONCE, RUN ANYWHERE”

INDÉPENDANCE DU MATÉRIEL ET DE L’OS

SÉCURITÉ

GESTION AUTOMATIQUE DE LA MÉMOIRE



BONNES PERFORMANCES (JIT COMPILER)

STANDARDISATION ET STABILITÉ

ÉCOSYSTÈME MULTI-LANGAGES

39

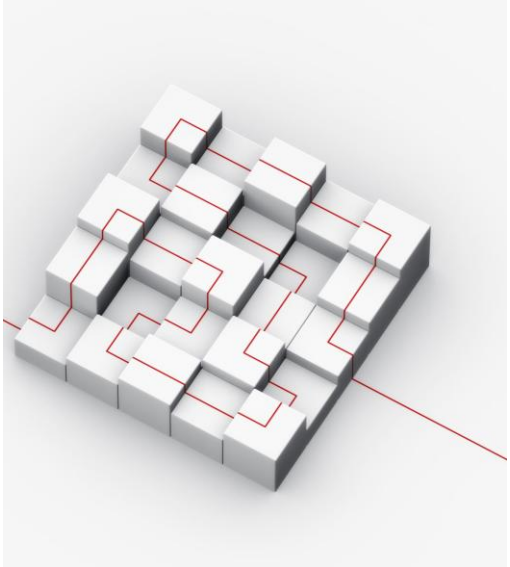
Cours 1 - Architecture JEE

Contenu

- Java EE / Jakarta EE
- Architecture en couches
- Rôle de chaque couche
- Comparaison JEE classique / Spring

Livrable cours

- Schéma d’architecture JEE
- Cas fil rouge : *Gestion de Bibliothèque*



40

Cours 2 — Maven & gestion des dépendances

Contenu

- Problèmes sans Maven
- pom.xml
- Cycle de vie Maven
- Dépendances Spring

41

Cours 3 — Architecture Spring Framework

- Core Container
- Data Access Integration
- Web
- Place de Spring Boot

42

Cours 4 — IoC & Injection de dépendances

- Principe IoC
- DI par constructeur
- Cycle de vie des beans
- Scopes (Singleton, Prototype)

43

Cours 5 — Design Patterns en JEE

- MVC
- Singleton
- Factory
- Repository
- Service Layer

44

Cours 6 — Spring MVC & REST

HTTP & REST

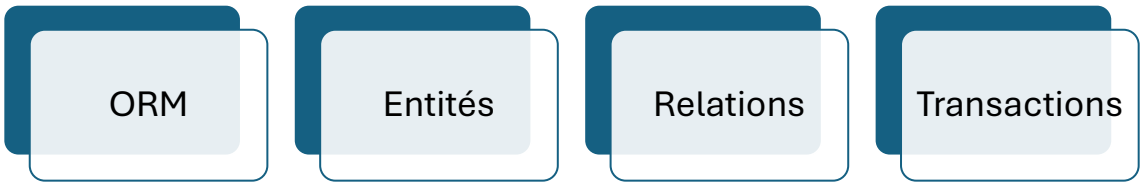
@Controller vs @RestController

Mapping URL

DTO

45

Cours 7 — Accès aux données avec JPA



46

Liste des TPs

Cours	TP
Architecture JEE	TP1
Maven	TP1
Spring Core / IoC	TP2
Design Pattern	TP3
Spring MVC	TP4
Spring Boot	TP5
REST	TP6
JPA	TP7–TP8
Services métiers	TP9
Synthèse	TP10

47

48